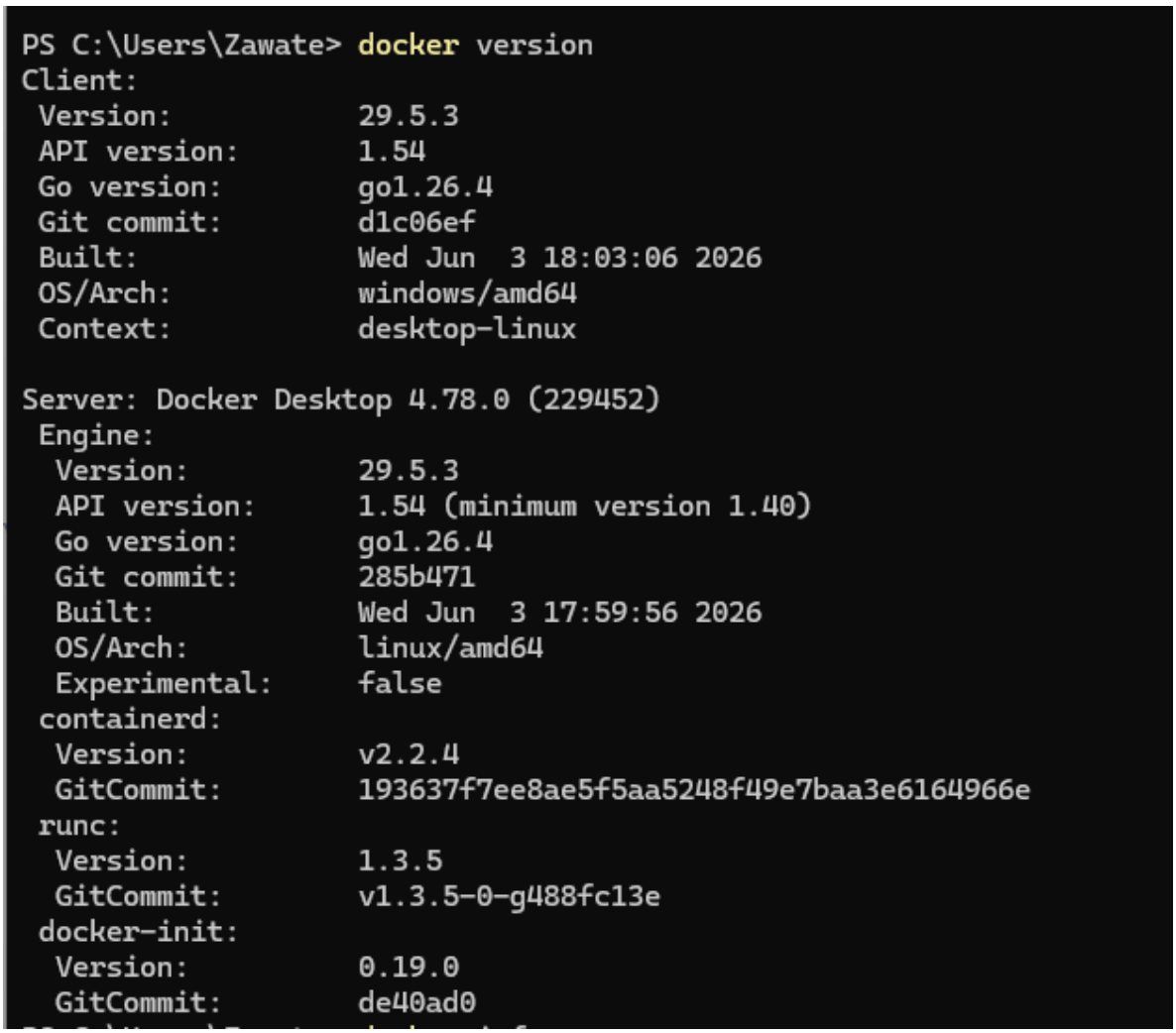
LABORATORIO 1: Fundamentos de Contenedores y Ciclo de Vida CLI

Objetivo del Laboratorio: Comprender la arquitectura cliente-servidor de Docker, desplegar contenedores

interactivos y gestionar su ciclo de vida mediante la línea de comandos.

Fase 1: Verificación de Arquitectura e Inspección del Sistema



```
PS C:\Users\Zawate> docker info
Client:
 Version:    29.5.3
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.73.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.25.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.34.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.1.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  dhi: CLI for managing Docker Hardened Images (Docker Inc.)
    Version:  v0.0.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-dhi.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.42.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.2.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.6.3
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.1.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: Docker Sandbox (Docker Inc.)
    Version:  v0.12.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sandbox.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.21.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-scout.exe

Server:
 Containers: 8
  Running: 1
  Paused: 0
  Stopped: 7
 Images: 14
 Server Version: 29.5.3
```

```
Storage Driver: overlayfs
 driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
 Volume: local
 Network: bridge host ipvlan macvlan null overlay
 Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
 /etc/cdi
 /var/run/cdi
Discovered Devices:
 cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: io.containerd.runc.v2 nvidia runc
Default Runtime: runc
Init Binary: docker-init
containerd version: 193637f7ee8ae5f5aa5248f49e7baa3e6164966e
runc version: v1.3.5-0-g488fc13e
init version: de40ad0
Security Options:
 seccomp
  Profile: builtin
 cgroupns
Kernel Version: 6.18.33.1-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 24
Total Memory: 30.87GiB
Name: docker-desktop
ID: 2e50254a-9810-4f08-bd9f-70cf70e8bcdf
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
 com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
 hubproxy.docker.internal:5555
 127.0.0.0/8
 ::1/128
Live Restore Enabled: false
Firewall Backend: iptables
```

Fase 2: Despliegue, Mapeo de Puertos e Interacción Directa

```
PS C:\Users\Zawate> docker run -d -p 8080:80 --name servidor-web-01 nginx
982ca88da375373ee2781ad4f62efb7478f60f87427901242494Ofb02733e7c4
PS C:\Users\Zawate> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
   NAMES
982ca88da375   nginx     "/docker-entrypoint.…"   6 seconds ago   Up 6 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
   servidor-web-01
PS C:\Users\Zawate> docker logs servidor-web-01
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/13 02:45:30 [notice] 1#1: using the "epoll" event method
2026/10/13 02:45:30 [notice] 1#1: nginx/1.31.2
2026/10/13 02:45:30 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/10/13 02:45:30 [notice] 1#1: OS: Linux 6.18.33.1-microsoft-standard-WSL2
2026/10/13 02:45:30 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/13 02:45:30 [notice] 1#1: start worker processes
2026/10/13 02:45:30 [notice] 1#1: start worker process 29
2026/10/13 02:45:30 [notice] 1#1: start worker process 30
2026/10/13 02:45:30 [notice] 1#1: start worker process 31
2026/10/13 02:45:30 [notice] 1#1: start worker process 32
2026/10/13 02:45:30 [notice] 1#1: start worker process 33
2026/10/13 02:45:30 [notice] 1#1: start worker process 34
2026/10/13 02:45:30 [notice] 1#1: start worker process 35
2026/10/13 02:45:30 [notice] 1#1: start worker process 36
2026/10/13 02:45:30 [notice] 1#1: start worker process 37
2026/10/13 02:45:30 [notice] 1#1: start worker process 38
2026/10/13 02:45:30 [notice] 1#1: start worker process 39
2026/10/13 02:45:30 [notice] 1#1: start worker process 40
2026/10/13 02:45:30 [notice] 1#1: start worker process 41
2026/10/13 02:45:30 [notice] 1#1: start worker process 42
2026/10/13 02:45:30 [notice] 1#1: start worker process 43
2026/10/13 02:45:30 [notice] 1#1: start worker process 44
2026/10/13 02:45:30 [notice] 1#1: start worker process 45
2026/10/13 02:45:30 [notice] 1#1: start worker process 46
2026/10/13 02:45:30 [notice] 1#1: start worker process 47
2026/10/13 02:45:30 [notice] 1#1: start worker process 48
2026/10/13 02:45:30 [notice] 1#1: start worker process 49
2026/10/13 02:45:30 [notice] 1#1: start worker process 50
2026/10/13 02:45:30 [notice] 1#1: start worker process 51
2026/10/13 02:45:30 [notice] 1#1: start worker process 52

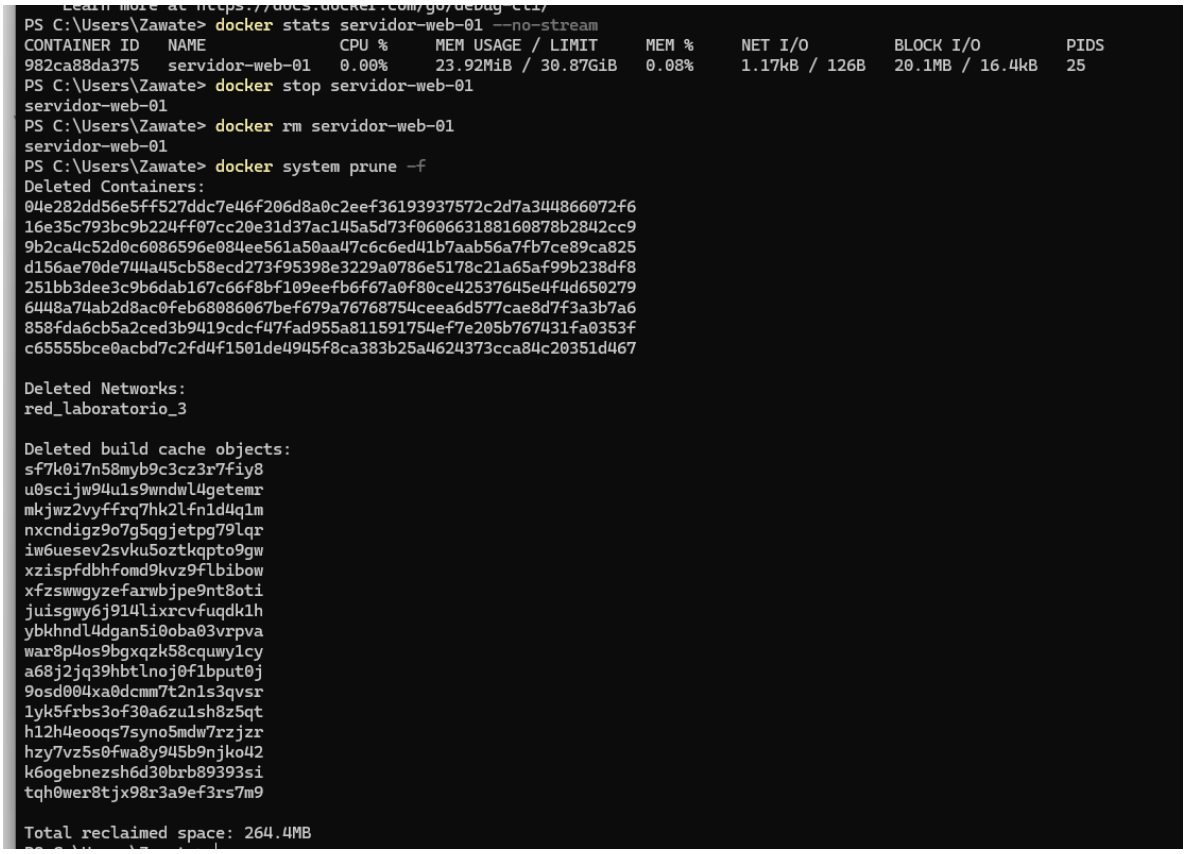
What's next:
    View and search logs for all containers in one place
    with Docker Desktop's Logs view. docker-desktop://dashboard/logs
```

```
PS C:\Users\Zawate> docker exec -it servidor-web-01 bash
root@982ca88da375:/# cat /etc/issue
Debian GNU/Linux 13 \n \l

root@982ca88da375:/# exit
exit

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug servidor-web-01
    Learn more at https://docs.docker.com/go/debug-cli/
```

Fase 3: Monitoreo de Recursos, Detención y Limpieza

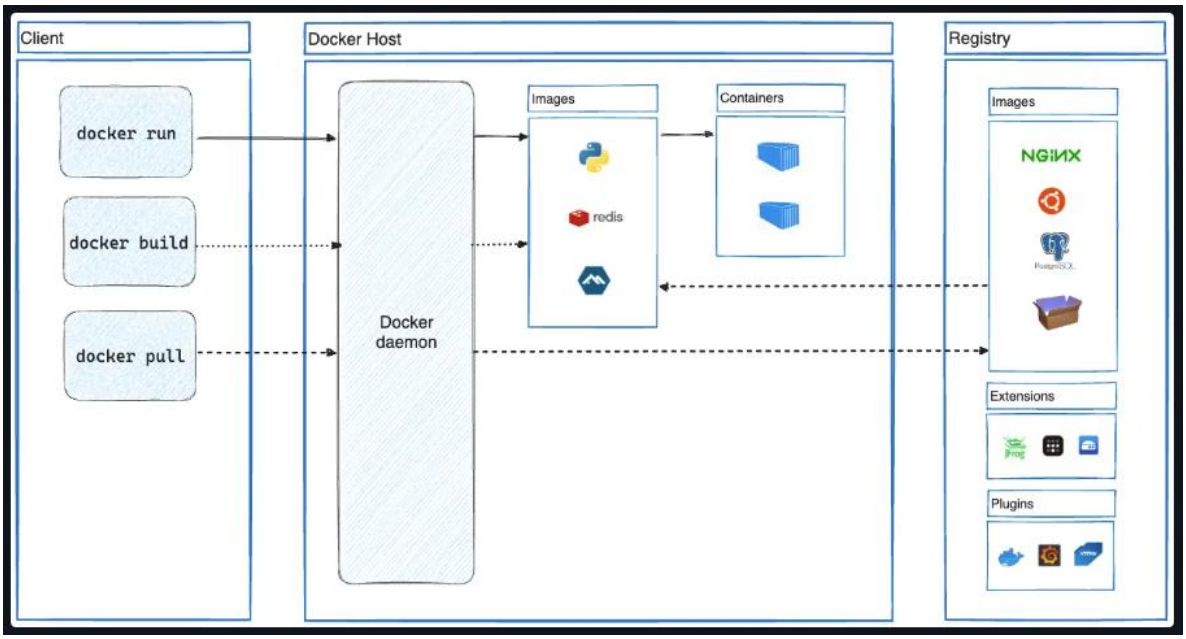


1. ¿Cuál es la función del motor (Daemon) de Docker frente al cliente ejecutable en la línea de comandos?

https://docs.docker.com/get-started/docker-overview/

https://kodekloud.com/blog/docker-architecture/

https://dev.to/juniourrau/docker-architecture-the-engine-behind-the-magic-4d2a



Docker utiliza una arquitectura de tipo cliente-servidor:

- **El Cliente (docker):** Es la interfaz de línea de comandos (CLI) con la que tú interactúas directamente. Su única tarea es recibir tus instrucciones (por ejemplo, docker run o docker ps) y **traducirlas en peticiones a una API REST**. El cliente no ejecuta, no construye, ni sabe cómo aislar un contenedor por sí mismo.
- **El Motor o Daemon (dockerd):** Es el servicio que corre de forma persistente en el fondo del sistema operativo anfitrión. Es el verdadero "cerebro" y fuerza de trabajo de Docker. Escucha las peticiones de la API REST que le envía el cliente y **realiza el trabajo pesado**: administrar imágenes, crear los entornos de red, gestionar el almacenamiento y orquestar el ciclo de vida de los contenedores.

2. Explique qué sucede internamente cuando se ejecuta la opción -p 8080:80 en el comando docker run.

https://www.youtube.com/watch?v=peMxZW3M24g

La opción -p (abreviación de *publish*) se encarga de habilitar el tráfico de red desde el exterior hacia el contenedor:

- **En el Contenedor (Puerto 80):** El proceso dentro del contenedor (por ejemplo, Nginx o Apache) está configurado para escuchar peticiones localmente en su puerto interno 80.
- **En tu Servidor/Computadora local (Puerto 8080):** El motor de Docker secuestra o reserva el puerto 8080 de tu máquina real para que nadie más lo use.
- **El mecanismo interno (Mapeo):** Tras bambalinas, el Daemon de Docker modifica la configuración del firewall de tu sistema operativo (creando reglas en iptables en Linux, o mediante el subsistema de red de Docker Desktop). Cuando llega tráfico web externo a la dirección http://localhost:8080, el firewall intercepta el paquete y lo redirige de forma transparente hacia la IP privada del contenedor directo al puerto 80.

3. ¿Qué diferencia existe entre detener un contenedor (docker stop) y eliminarlo (docker rm)?

https://medium.com/@golibraryco/lifecycle-management-for-docker-containers-create-run-pause-stop-and-delete-68bd7476a0f6

https://stackoverflow.com/questions/33362856/whats-the-difference-between-docker-stop-and-docker-rm

El ciclo de vida de un contenedor separa su estado de ejecución de su existencia en el disco duro:

- **docker stop (Detener):** Envía primero una señal limpia de apagado (SIGTERM) al proceso principal del contenedor para que guarde cambios y cierre conexiones de forma ordenada. Si no responde tras 10 segundos, lo fuerza a apagarse con un SIGKILL. El contenedor pasa al estado Exited (Salido). **Los datos de su capa de escritura y su configuración siguen intactos en el disco**. Puedes volver a encenderlo tal y como estaba usando docker start.
- **docker rm (Eliminar):** Borra por completo el contenedor del sistema. Esto significa que **destruye su metatada y su capa de almacenamiento escribible** de forma permanente. No puedes volver a iniciarlo. Para recuperar el servicio tendrás que crear un contenedor completamente nuevo desde cero

utilizando la imagen base. *(Nota: Los volúmenes compartidos no se eliminan con este comando a menos que uses la bandera -v)*.